WEEK 4 · LESSON 22 OF 24 · HANDOUT

Autoencoders and Generative Models

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=bbOFvxbMIV0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroSciPy: Deep Diving into GANs — from Theory to Production — https://www.youtube.com/watch?v=CePrdabdtxw

Your goal in this lesson: Build an autoencoder that compresses images into a tiny code, use it to detect unusual inputs and to create new images, and explain how VAEs, GANs and diffusion models generate data.

Learning without labels — again

An **autoencoder** is a network trained to **copy its input to its output** through a narrow middle layer called the **bottleneck**:

> *image → encoder → small code → decoder → rebuilt image*

Because the code is small, the network cannot memorise every pixel. It must learn the most important patterns — a kind of non-linear PCA (Machine Learning Fundamentals, Lesson 20). No labels are needed: the image is its own target.

An autoencoder for digits

We squeeze each 64-pixel digit into just **2 numbers**:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
X = torch.tensor(digits.data / 16.0, dtype=torch.float32)
labels = digits.target

torch.manual_seed(0)
encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 2))
decoder = nn.Sequential(nn.Linear(2, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
opt = torch.optim.Adam(list(encoder.parameters()) + list(decoder.parameters()), lr=1e-3)

for epoch in range(100):
    perm = torch.randperm(len(X))
    for start in range(0, len(perm), 64):
        idx = perm[start:start + 64]
        opt.zero_grad()
        loss = F.mse_loss(decoder(encoder(X[idx])), X[idx])   # target = the input itself
        loss.backward()
        opt.step()
print("reconstruction error:", round(loss.item(), 4))
```

Look at the rebuilt digits

```
with torch.no_grad():
    rebuilt = decoder(encoder(X[:8]))
fig, axes = plt.subplots(2, 8, figsize=(10, 3))
for k in range(8):
    axes[0, k].imshow(X[k].view(8, 8), cmap="gray_r"); axes[0, k].axis("off")
    axes[1, k].imshow(rebuilt[k].view(8, 8), cmap="gray_r"); axes[1, k].axis("off")
axes[0, 0].set_title("original", loc="left"); axes[1, 0].set_title("from 2 numbers",
loc="left")
plt.show()
```

From only two numbers the decoder rebuilds blurry but recognisable digits.

The map of the code space

```
with torch.no_grad():
    codes = encoder(X).numpy()
plt.figure(figsize=(6, 5))
plt.scatter(codes[:, 0], codes[:, 1], c=labels, cmap="tab10", s=6)
plt.colorbar(label="digit")
plt.title("Every digit as a point in the 2-D code space")
plt.show()
```

Digits of the same kind gather in the same regions — although the autoencoder **never saw a label**.

Use 1 — Anomaly detection

An autoencoder rebuilds well only what looks like its training data. Anything unusual comes back badly:

```
with torch.no_grad():
    digit_error = ((decoder(encoder(X)) - X) ** 2).mean(dim=1)
    noise = torch.rand(20, 64)
    noise_error = ((decoder(encoder(noise)) - noise) ** 2).mean(dim=1)
print("average error, real digits: ", round(digit_error.mean().item(), 3))
print("average error, random noise:", round(noise_error.mean().item(), 3))
```

Random noise has an error about **five times** larger. The same trick flags unusual machine vibrations, network traffic or transactions — without any labelled examples of failures.

Use 2 — Generating new images

Pick new points in the code space and decode them: images that were **never in the dataset**.

```
grid = torch.tensor([[a, b] for b in np.linspace(codes[:, 1].max(), codes[:, 1].min(), 6)
                            for a in np.linspace(codes[:, 0].min(), codes[:, 0].max(), 6)],
                    dtype=torch.float32)
with torch.no_grad():
    made = decoder(grid)
fig, axes = plt.subplots(6, 6, figsize=(6, 6))
for ax, img in zip(axes.ravel(), made):
    ax.imshow(img.view(8, 8), cmap="gray_r")
    ax.axis("off")
plt.suptitle("New digits decoded from points on a grid")
plt.show()
```

Moving across the grid morphs one digit smoothly into another. A plain autoencoder's code space has gaps, though, so some points decode to nonsense.

The generative model family

Model	Idea	Known for
VAE (variational autoencoder)	Forces the code space to be smooth and filled, so any point decodes to something sensible	Smooth latent spaces, science data
GAN (generative adversarial network)	A generator fakes images; a discriminator tries to catch fakes; both improve	Very sharp faces (2014–2020)
Diffusion model	Learns to remove noise step by step, starting from pure noise	Stable Diffusion, DALL-E, video models
Autoregressive transformer	Predicts the next token again and again	ChatGPT, Claude, music and code generation

Responsibility

Generative models can create **deepfakes**, fake documents and misleading images. Label AI-generated content, never generate content that imitates real people or official documents, and respect the rights of the people whose data trained the model.

Key terms

- **Autoencoder** — a network trained to rebuild its input through a bottleneck.
- **Encoder / decoder** — compresses to a code / rebuilds from a code.
- **Latent (code) space** — the space of compressed codes.

- **Reconstruction error** — how badly the input is rebuilt; high for anomalies.
- **VAE, GAN, diffusion** — families of generative models.

Remember this: Autoencoders learn compact codes without labels; their reconstruction error spots anomalies, and decoding new codes generates new data — the doorway to modern generative AI.

Name: _____ Date: _____

Activity

Task 1

Use a bottleneck of 8 instead of 2. How do the reconstructions and the error change?

Task 2

Train the autoencoder only on digits 0–4, then compare the reconstruction error of digits 5–9. Could this detect "new kinds" of data?

Task 3

Add noise to the inputs but keep clean targets (a **denoising** autoencoder). Show noisy inputs and the cleaned outputs.

Task 4

In two sentences, explain to a friend how a diffusion model makes an image.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is an autoencoder trained to do?
   - A. Rebuild its own input through a small bottleneck
   - B. Sort images
   - C. Translate text
   - D. Predict labels
2. Why must the bottleneck be small?
   - A. To save electricity only
   - B. To force the network to learn the most important patterns instead of copying
   - C. To add labels
   - D. Because GPUs require it
3. How can an autoencoder detect anomalies?
   - A. It counts pixels
   - B. It cannot
   - C. Unusual inputs are rebuilt badly, so their reconstruction error is high
   - D. It uses labels of failures
4. How can a trained decoder generate new images?
   - A. By adding noise to labels
   - B. By decoding new points from the code space
   - C. By copying training images
   - D. By sorting labels
5. Which model family powers tools like Stable Diffusion?
   - A. k-means
   - B. Diffusion models that remove noise step by step
   - C. LSTMs only
   - D. Decision trees
6. In a GAN, what does the discriminator do?
   - A. Sets the learning rate
   - B. Generates images
   - C. Encodes text
   - D. Tries to tell real images from generated ones

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.